EEE 485/585 Project Report

Berkin Serim

22302434

1 Problem and Data Set

1.1 The Problem and Expected Challenges

The primary goal of this project is to classify left-hand and right-hand brain signals using EEG data. The main challenge is creating a model that works for everyone, which is why we train our system on 85 subjects and test it on 21 new people it has never seen. This cross-subject approach is difficult because individual differences like skull thickness and electrode impedance create unique noise for every person. We also compare motor imagery, where a person only thinks about a movement, with motor execution, where they actually move. Motor imagery is expected to be harder to classify because it produces a weaker event-related desynchronization (ERD) than motor execution.

1.2 Data Set and Preprocessing

The PhysioNet EEG Motor Movement/Imagery Dataset is used [1, 2, 3]. We downloaded and read the original EDF files with the MNE-Python library [4]. Even though the dataset has Baseline (eyes open, eyes closed), Open and Close (left fist, right fist, both fists, both feet), and Imagine Opening and Closing (left fist, right fist, both fists, both feet), only the Imagine and actual Opening and Closing (left and right fist) will be used. The dataset for this project consists of multi-channel EEG recordings from 109 subjects; however, three subjects (88, 92, and 100) were excluded because their recordings were not sampled at 160 Hz. The remaining 106 subjects were partitioned into a training pool of 85 subjects and a test set of 21 subjects. To streamline the data for classification, we selected 21 electrodes surrounding the C_3, C_z, and C_4 positions by their names (FC5, FC3, FC1, FCz, FC2, FC4, FC6, C5, C3, C1, Cz, C2, C4, C6, CP5, CP3, CP1, CPz, CP2, CP4, CP6), which corresponds to motor cortex [5]. This localized our analysis to the sensorimotor cortex and effectively ignored noise from eye movements or visual processing in other areas. Following the spatial selection, we passed the signals (sampled at 160 Hz) through a 5th-order Butterworth bandpass filter targeting the 8–30 Hz frequency range [6], applied forward and backward so that it does not shift the signal in time (zero-phase).

Once filtered, we processed every trial to remove DC offsets by calculating the temporal mean for each channel and subtracting it from the signal, ensuring our data was centered at zero. We then implemented trace normalization by dividing the covariance of each trial by its own trace. This was done to equalize the signal power across different subjects and tasks, preventing a subject with naturally higher scalp conductivity from skewing the global model. We applied these identical signal processing steps to both the motor imagery and the motor execution datasets to ensure mathematical consistency. Finally, the CSP features were extracted by solving the generalized eigenvalue problem using scipy.linalg.eigh [7]. We specifically chose this function over a standard eigenvalue solver because it is optimized for the symmetric positive-definite matrices generated by our EEG covariance data, ensuring numerical stability. While the model was trained exclusively on motor imagery trials, motor execution data was included in the testing phase to evaluate how well the system generalizes to actual movements.

2 Review of Machine Learning Algorithms

2.1 k-NN

We selected k-NN as our primary baseline because its non-parametric nature allows us to evaluate the raw discriminative power of the CSP features without the influence of an iterative training process. The k-NN algorithm operates on the principle of feature-space proximity, assuming that trials with similar spatial variance characteristics (CSP features) belong to the same class of motor imagery. In our implementation, every trial is represented as a point in a $2m$-dimensional space. To classify a new, unseen trial from the test set, the algorithm calculates the distance between that test point and every single point in the training set. We utilized Euclidean distance as our proximity metric, defined as:

$$d(x, q) = \sqrt{\sum_{i=1}^{n}(x_i - q_i)^2}$$

where q is the vector of test characteristics and x is a training vector. Once all distances are computed, the algorithm identifies the "k" closest training examples. The final classification is decided through a Majority Voting mechanism: the test trial is assigned to the class (Left or Right hand) that appears most frequently among its k nearest neighbors.

A unique characteristic of k-NN in this project is that it has no explicit Loss Function or optimization phase. While Logistic Regression uses Gradient Descent to minimize Cross-Entropy error, k-NN is a "lazy learner" that performs all computations at the time of prediction. The "learning" in k-NN is effectively replaced by the selection of the Hyperparameter k. We decided on the optimal k value by evaluating the model's performance over a range of odd integers (to avoid tie votes) during the cross-validation phase. A small k (e.g., $k = 1$) makes the model highly sensitive to noise and outliers in the EEG data, while a very large k might smooth over the distinct boundaries between the motor imagery classes. Thus, k-NN serves as a vital baseline to determine if the CSP features are naturally clustered before we apply more complex, weight-based optimization models.

2.2 Logistic Regression

Unlike k-NN, which requires the entire dataset for every prediction, Logistic Regression learns a specific weight vector w and a bias term b during the training phase. The model functions by taking the linear combination of the CSP features ($z = w \cdot x + b$) and passing it through a Sigmoid Activation Function, defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function maps any real-valued input into a probability range between 0 and 1, representing the likelihood that a given EEG trial belongs to the "Right Hand" class. We implemented the training process from scratch using Full Batch Gradient Descent. To find the optimal parameters, the algorithm iteratively minimizes the Regularized Binary Cross-Entropy Loss $J(w, b)$:

$$J(w, b) = -\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1 - y^{(i)})\log(1 - \hat{y}^{(i)})\right] + \frac{\lambda}{2N}\sum w^2$$

The L2 term (λ) adds a penalty to the weights, preventing the model from assigning excessively high importance to subject-specific noise. In our final implementation we set $\lambda = 0$, since the CSP feature space is small relative to the number of trials (Section 3.4). The optimization is performed by updating the weights and bias at each iteration using the learning rate $\alpha = 0.1$:

$$w := w - \alpha \cdot \frac{\partial J}{\partial w}, \quad b := b - \alpha \cdot \frac{\partial J}{\partial b}$$

Training was run for a fixed number of 2,000 iterations.

2.3 Linear Discriminant Analysis (LDA)

LDA is the mathematical sibling of CSP. Since CSP finds the directions that maximize variance differences, and LDA finds the projection that maximizes class separation based on those variances, they are a "perfect pair." LDA was chosen because it provides a closed-form solution (no iterations). While Logistic Regression is a discriminative model that minimizes error, LDA is a generative model that seeks to project the CSP features into a lower-dimensional space that maximizes the distance between class means while minimizing the spread within each class. This approach is theoretically ideal for CSP features, which are specifically designed to create distinct variance-based "clouds" for different motor imagery tasks.

The algorithm functions by estimating the Centroids (mean vectors) for the Left Hand (μ_0) and Right Hand (μ_1) classes, along with a shared Pooled Covariance Matrix (Σ). The decision boundary is determined by a weight vector w that represents the linear combination of features yielding the maximum class separation. Mathematically, the weights are calculated using the inverse of the covariance matrix:

$$w = \Sigma^{-1}(\mu_1 - \mu_0)$$

Because brain signals are inherently noisy, a direct inversion of the covariance matrix can lead to numerical instability or overfitting. To counter this, we implemented Shrinkage LDA (Regularization). We modified the covariance matrix by blending it with an identity matrix I using a shrinkage parameter $\alpha = 0.1$:

$$\Sigma_{reg} = (1 - \alpha)\Sigma + \alpha I$$

The final classification for a test trial x is performed by calculating the projection $y = w \cdot x + b$. If the resulting value is positive, the trial is classified as a Right Hand movement; otherwise, it is assigned to the Left Hand class. By using this closed-form mathematical solution instead of an iterative loop, LDA provided a highly efficient and stable baseline for our cross-validation tests.

3 Implementation

3.1 Preprocessing

We initialized the process by establishing a strict subject-independent split. Using a fixed random seed (2601) for shuffling the 106 valid subjects, we allocated 85 subjects for training and 21 subjects (20%) for testing.

Within the data loading loop, we selected the 21 channels listed in Section 1.2 by their names. By filtering out the other 43 channels at the loading stage, we reduced the program's memory footprint and ensured that subsequent temporal filters were applied only to task-relevant cortical areas. The preprocessing then performed a two-step temporal cleanup. First, we subtracted each channel's mean to center the signal at zero. Second, we designed a 5th-order Butterworth bandpass filter using scipy.signal.butter and applied it with scipy.signal.sosfiltfilt, which filters the signal forward and backward so that no time shift (phase distortion) is added. The choice of a 5th-order filter provides a steep roll-off to effectively eliminate low-frequency drift and high-frequency muscle artifacts (EMG), while the 8–30 Hz range isolates the specific Mu and Beta rhythms associated with hand movement.

A significant part of our implementation involved custom epoch extraction in the extract_trials function. For each left-hand (T1) or right-hand (T2) cue, we cut the signal from 0.5 s to 2.5 s after the cue onset (320 samples). We skipped the first 0.5 s because it mostly contains the brain's response to seeing

the cue, while the ERD related to the movement develops after it. This gave 3,823 training trials and 945 test trials for both imagery and execution, and the two classes were balanced (49.5–49.8% right hand).

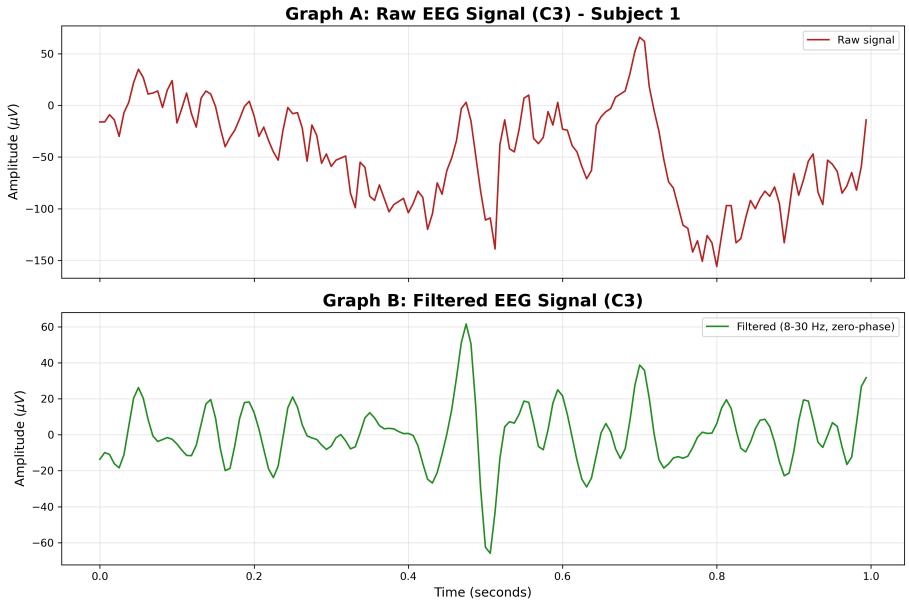


Figure 1: Comparison Between Not Preprocessed and Preprocessed Data (electrode C3, Subject 1)

Graph A shows the raw EEG from electrode C3 of Subject 1. Raw EEG contains a DC offset and slow drifts caused by electrode-skin interference and slow body movements. This must be fixed because the classification algorithms cannot distinguish this slow drift from actual brain activity.

Graph B shows the processed signal. It is now centered at zero and the drift has been removed. By applying the 8–30 Hz filter, we have stripped away the noise, leaving only the rhythmic Mu and Beta waves needed for the motor imagery task. This step is mandatory to ensure the model focuses on the user's intent rather than hardware or environmental noise.

3.2 Common Spatial Pattern (CSP)

The Common Spatial Patterns (CSP) algorithm was implemented as our primary feature extraction engine to identify the spatial filters that maximize signal variance for one motor imagery class while minimizing it for the other. This approach is specifically designed to capture Event-Related Desynchronization (ERD), where the power of the Mu and Beta bands fluctuates during the imagination of hand movements. By transforming the 21-channel EEG data into a new coordinate system defined by these filters, we effectively isolate the task-related neural firing patterns from the background brain activity.

The process involved calculating the average covariance matrices R_0 and R_1 for the Left and Right classes across the training subjects, where each trial covariance was first divided by its trace. We then solved the generalized eigenvalue problem to find the spatial filters W and eigenvalues λ:

$$R_0 W = \lambda(R_0 + R_1)W$$

We selected the m first and m last eigenvectors—representing the most discriminative spatial distributions—to form a transformation matrix W_m. Each trial E was then projected onto these filters, and the

log-variance of the resulting signals $Z = W_m^T E$ was calculated to serve as the final feature vector f_p:

$$f_p = \log\left(\frac{var(Z_p)}{\sum_{i=1}^{2m} var(Z_i)}\right)$$

This log-transformation is a critical step to linearize the power distribution of the EEG, ensuring the extracted features are suitable for the subsequent classifiers.

3.3 k-NN

The implementation of our classification pipeline followed a rigorous two-stage process: a 5-fold Cross-Validation grid search for optimization, followed by a final evaluation against the 21-subject lockbox set. In the first stage, we split the 85 training subjects into 5 folds of 17 subjects, so that all trials of a subject are always in the same fold. This way, the validation subjects are never seen during training, just like the test subjects. For each fold, we recalculated the CSP spatial filters from scratch, using only the training data to avoid data leakage. We performed a 2D Grid Search over two critical hyperparameters: the number of CSP filter pairs (m), ranging from 1 to 10 (with 21 channels there are at most 21 filters, so $2m \leq 21$), and the number of k-NN neighbors (k), taking odd values from 1 to 99. By averaging the validation accuracy across all folds, we identified the "Winning Parameters" that yielded the most stable performance across validation folds.

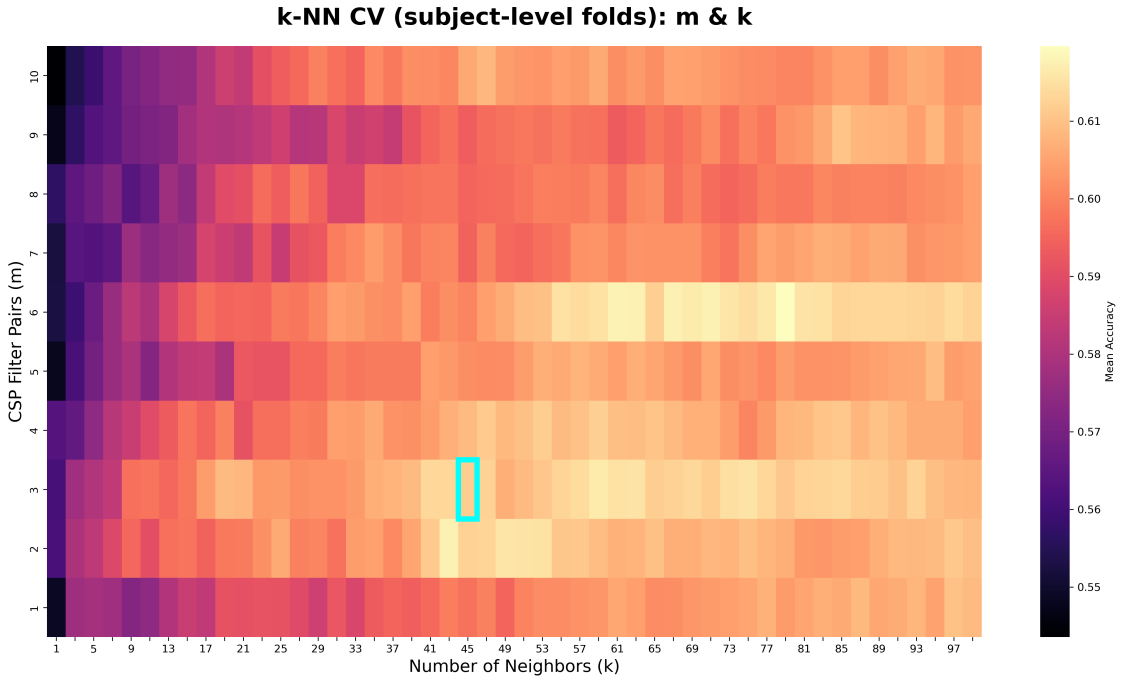


Figure 2: 2D Grid Search Heat Map

The hyperparameter heatmap (Figure 2) illustrates the average validation accuracy across the grid. The accuracy is low for small k values and reaches a plateau of about 60–62% for $k \geq 41$. Large k values work better because the neighbors come from other subjects and are noisy; averaging over many neighbors reduces this noise. In our selection process, we didn't just pick the highest number; we looked for a region where the accuracy remained consistent even if k or m shifted slightly, ensuring the model would be robust. Specifically, we averaged each cell of the heatmap with its 8 neighbors (a 3×3 moving average) and selected the cell with the highest smoothed accuracy. This gave $m = 3$ and $k = 45$ (indicated by the cyan box), with a mean validation accuracy of 61.18% (the single highest cell was 61.97%).

Once the optimal hyperparameters were identified, we moved to the Final Master Model stage. Instead of using a subset, we utilized all 85 training subjects to compute the master covariance matrices and solve the final Generalized Eigenvalue Decomposition (GEVD). These master spatial filters were then used to extract features from the 21-subject test set, which had remained completely untouched throughout the optimization process. The final classification was performed using our knn_predict_multi function, which calculates the Euclidean distance between the test trials and the entire 85-subject training base. This subject-independent approach ensures that the final project accuracy reflects the system's ability to generalize to new users, a key requirement for real-world BCI applications.

Motor Imagery (MI) Results

The model achieved an accuracy of 65.50%, correctly predicting 619 out of 945 trials. We calculated the Z-score to measure significance :

$$Z = \frac{619 - 472.5}{15.37} \approx 9.53$$

A Z-score of 9.53 indicates that performance is significantly above chance ($p < 0.001$).

Motor Execution (ME) Results

We applied the same procedure to classify actual movements. The model achieved an accuracy of 63.17%, correctly predicting 597 out of 945 trials. The Z-score for this condition is:

$$Z = \frac{597 - 472.5}{15.37} \approx 8.10$$

This Z-score is also significant ($p < 0.001$), indicating that the imagery-trained model transfers to physical actions.

Further Analysis

To further demonstrate that our results are not due to random guessing, we evaluate performance using a Binomial Test. Under the null hypothesis (H_0), the probability of success is $p_0 = 0.5$. For $n = 945$ trials, we calculate the expected mean (μ) and standard deviation (σ) to determine the Z-score:

$$\mu = n \cdot p_0 = 945 \cdot 0.5 = 472.5$$

$$\sigma = \sqrt{n \cdot p_0 \cdot (1 - p_0)} = \sqrt{945 \cdot 0.25} \approx 15.37$$

The Z-score represents how many standard deviations our observed result ($k = 619$, the k-NN motor imagery result) is from the mean:

$$Z = \frac{k - \mu}{\sigma} = \frac{619 - 472.5}{15.37} \approx 9.53$$

The exact one-sided binomial p-value is approximately 5×10^{-22}. Since $p < 0.05$, we reject the null hypothesis.

The binomial test treats all trials as independent, but trials from the same subject are related. Therefore, we also tested the 21 per-subject accuracies against 50% with a one-sample t-test. All three models were above chance on both test sets ($t(20) \geq 4.35$, $p < 0.001$).

Discussion

The model was 2.33 percentage points less accurate on execution than on imagery. A paired Wilcoxon signed-rank test over the 21 test subjects shows that this difference is not statistically significant ($p = 0.55$). We expected execution to be easier, because motor execution produces a stronger and more consistent ERD than motor imagery; this was not observed. One possible reason is that the model was trained only on imagery trials, so it is tuned to the imagery patterns.

Interestingly, the test accuracy (65.50%) is higher than the validation accuracy (61.18%), while usually the opposite is expected. We see two reasons for this. First, the final model was trained on all 85 subjects, while each cross-validation model was trained on only 68 subjects. Second, the accuracy changes a lot from subject to subject (standard deviation of about 15 percentage points), so with only 21 test subjects the average has a standard error of about 3.4 points ($15.4/\sqrt{21}$), and our test subjects may simply be easier on average. This highlights the "Intersubject Variability" problem. EEG signals are highly specific to an individual's skull thickness, electrode placement, and neural anatomy. Furthermore, biological noise such as "non-stationarity"—where a subject's brain signals change due to fatigue or loss of focus over time—creates a shifting baseline that a static CSP model cannot fully compensate for. These factors constitute a "noise floor" in EEG engineering that limits accuracy but does not invalidate the pipeline's statistical success.

3.4 Logistic Regression

We implemented a 1-D Grid Search over $m \in [1, 10]$ using the same 5-fold subject-level cross-validation on the 85-subject training pool. The custom Logistic Regression model used a full batch gradient descent optimizer (learning rate 0.1, 2,000 iterations) without explicit regularization.

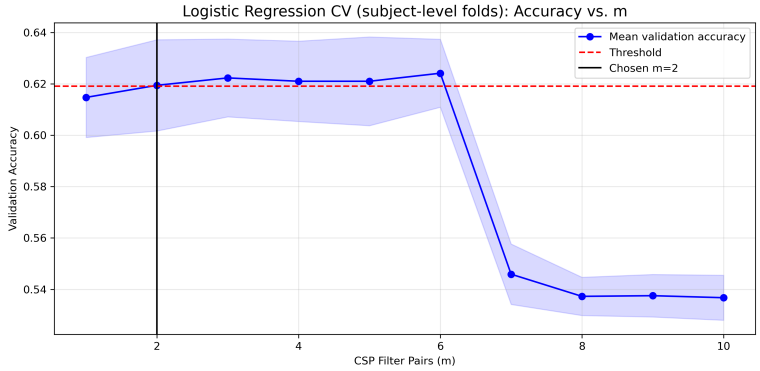


Figure 3: Logistic Regression Grid Search on m

The "winning parameter" was identified as $m = 2$, as shown in Figure 3. Our threshold for selecting this parameter was set to the smallest m whose mean validation accuracy is within 0.005 of the maximum (61.94% at $m = 2$, compared to the maximum of 62.41%). We did not need explicit regularization because our data size is much larger than our number of predictors ($N \gg p$); we have thousands of training trials to fit only 4 features ($2m$). Because the feature space is so small relative to the number of observations, the model naturally resists overfitting. Keeping m low prevents the algorithm from fitting to subject-specific artifacts and ensures it focuses strictly on universal motor patterns.

Motor Imagery (MI) Results

The final evaluation of the Logistic Regression master model on the 21-subject test set yielded an accuracy of 68.15%, correctly predicting 644 out of 945 trials. To verify this result, we used a Binomial Test

with a null hypothesis of $p = 0.5$ ($\mu = 472.5, \sigma = 15.37$). We calculated the Z-score as :

$$Z = \frac{644 - 472.5}{15.37} \approx 11.16$$

This Z-score of 11.16 indicates that the performance is statistically significant ($p < 0.001$).

Motor Execution (ME) Results

We applied the same model to classify actual motor movements to test its generalization. The model achieved an accuracy of 63.81%, correctly predicting 603 out of 945 trials. The Z-score for this condition is :

$$Z = \frac{603 - 472.5}{15.37} \approx 8.49$$

The Z-score for Motor Execution suggests that the patterns learned from imagery trials are also recognizable in execution trials.

Discussion

The Logistic Regression model was 4.34 percentage points less accurate on execution than on imagery, following the same direction as the k-NN model. However, a paired Wilcoxon signed-rank test shows that this difference is not statistically significant ($p = 0.22$).

The fact that both the k-NN and Logistic Regression models were less accurate on the execution test set, although not significantly, suggests that the CSP filters learned from imagery transfer to actual movement, but not perfectly.

3.5 Linear Discriminant Analysis (LDA)

The implementation of our classification pipeline followed a rigorous two-stage process: a 1-D Grid Search over $m \in [1, 10]$ for optimization using the same 5-fold subject-level cross-validation on the 85-subject training pool, followed by a final evaluation against the 21-subject "lockbox" test set. The Linear Discriminant Analysis (LDA) model calculates class centroids and a pooled covariance matrix to find the optimal linear decision boundary separating the left and right-hand motor imagery classes.

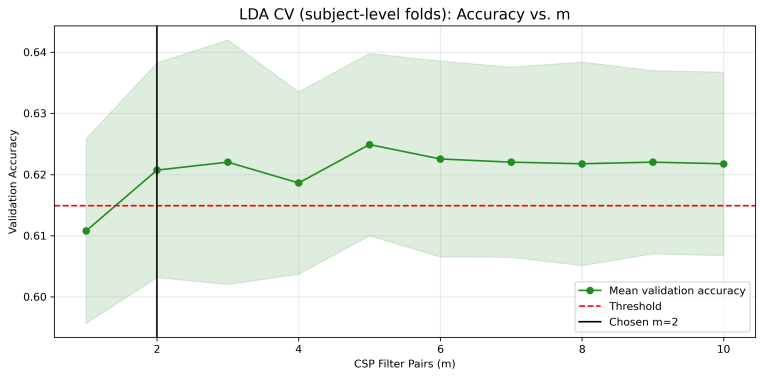


Figure 4: LDA Grid Search on m

As seen in Figure 4, we selected $m = 2$ as our optimal parameter. This choice was based on selecting the smallest m whose mean validation accuracy is within 0.01 of the maximum (62.07% at $m = 2$, compared to the maximum of 62.49%). Apart from the light shrinkage ($\alpha = 0.1$) described in Section 2.3, we did not use additional regularization in this model. This decision was made because our data size is

much larger than our number of predictors ($N \gg p$); we have thousands of training trials to optimize only 4 features ($2m$). Because the feature space is so small relative to the number of observations, the model is naturally resistant to overfitting.

Motor Imagery (MI) Results

The model achieved an accuracy of 68.36%, correctly predicting 646 out of 945 trials. We calculated the Z-score to measure significance :

$$Z = \frac{646 - 472.5}{15.37} \approx 11.29$$

A Z-score of 11.29 indicates performance significantly above chance ($p < 0.001$).

Motor Execution (ME) Results

We applied the same LDA model to classify actual motor movements to evaluate cross-condition generalization. The model achieved an accuracy of 64.44%, correctly predicting 609 out of 945 trials. The Z-score for this condition is :

$$Z = \frac{609 - 472.5}{15.37} \approx 8.88$$

This Z-score for Motor Execution suggests that the spatial patterns learned from the imagery training data are also present during the actual physical execution of the movement.

Discussion

The LDA model was 3.92 percentage points less accurate on execution than on imagery; this difference is also not statistically significant (paired Wilcoxon signed-rank test, $p = 0.38$). LDA reached the highest accuracy among our three architectures on both test sets (68.36% on MI and 64.44% on ME). However, the difference to Logistic Regression is very small (68.15% on MI, only 2 trials), while k-NN was about 3 percentage points lower. Both linear models work well with CSP features, and LDA is mathematically designed to find the most stable linear boundary in Gaussian noise. An algorithm trained entirely on the imagery trials of 85 subjects predicted the executed movements of 21 completely unseen subjects with 64.44% accuracy.

3.6 Comparison with a Reference Implementation

To check our own CSP and LDA code, we ran the same data split with the CSP implementation of MNE-Python [4] and the LDA implementation of scikit-learn [8], using their default settings and the same number of filters ($2m = 4$). This reference pipeline reached 63.92% on imagery and 63.07% on execution, which is comparable to our results (68.36% and 64.44%) and indicates that our implementation works as expected. The default settings of the reference pipeline handle normalization and filter selection differently from our implementation (Section 1.2), which may explain the small difference; we did not test this directly.

3.7 Per-Subject Analysis

Figure 5 shows the accuracy for each of the 21 test subjects. The accuracy changes a lot between subjects: for some subjects it is above 90% (e.g., Subject 7), while for others (e.g., Subjects 67 and 97) it stays close to chance level. This is a known problem in BCI research: some users do not produce a motor imagery pattern that can be classified (often called BCI inefficiency). Because of this variability, the average accuracy depends strongly on which subjects are in the test set, as discussed in Section 3.3.

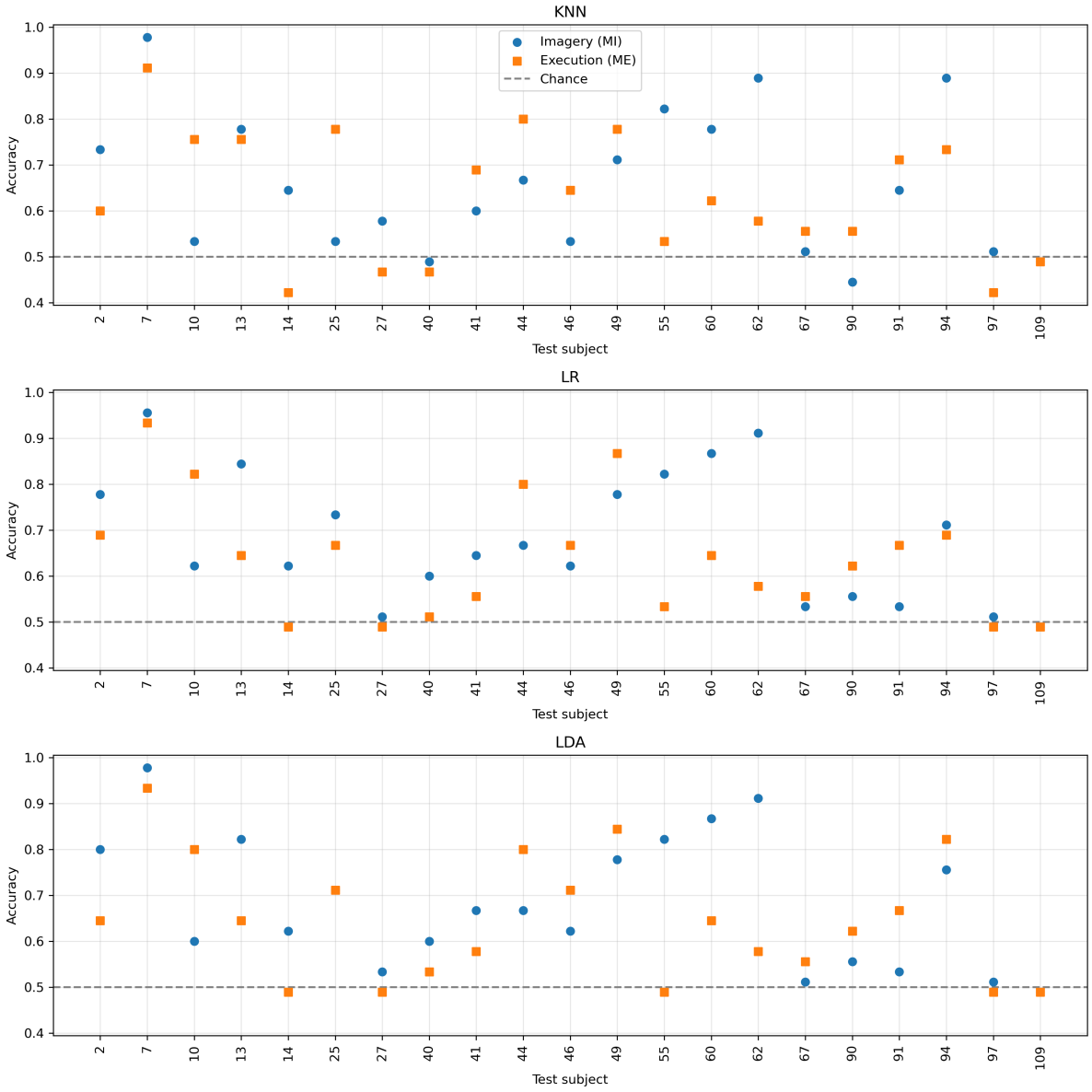


Figure 5: Accuracy of each test subject for the three models

References

[1] G. Schalk, "EEG motor movement/imagery dataset (version 1.0.0)." PhysioNet, 2009. RRID:SCR_007345. doi: 10.13026/C28G6P.

[2] G. Schalk, D. J. McFarland, T. Hinterberger, N. Birbaumer, and J. R. Wolpaw, "BCI2000: A general-purpose brain-computer interface (BCI) system," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 6, pp. 1034–1043, 2004.

[3] T. Pollard, B. E. Moody, L. Lehman, B. Gow, C. Fernandes, C. Xie, A. Johnson, R. G. Mark, and T. Heldt, "PhysioNet as a global platform for biomedical research," *Nature Health*, 2026. doi: 10.1038/s44360-026-00096-z.

[4] A. Gramfort, M. Luessi, E. Larson, D. A. Engemann, D. Strohmeier, C. Brodbeck, R. Goj, M. Jas, T. Brooks, L. Parkkonen, and M. Hämäläinen, "MEG and EEG data analysis with MNE-Python," *Frontiers in Neuroscience*, vol. 7, p. 267, 2013. doi: 10.3389/fnins.2013.00267.

[5] G. Pfurtscheller and C. Neuper, "Motor imagery and direct brain-computer communication," *Proceedings of the IEEE*, vol. 89, no. 7, pp. 1123–1134, 2001.

[6] G. Pfurtscheller and F. L. Da Silva, "Event-related eeg/meg synchronization and desynchronization: basic principles," *Clinical neurophysiology*, vol. 110, no. 11, pp. 1842–1857, 1999.

[7] "Lecture 7.3 common spatial patterns." https://youtu.be/zsOULC16USU?si=iS4uQWWrZDaAhbK, $Aug.2013$.

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Appendix

Code 1 (utilities.py)

```
import numpy as np
from scipy.linalg import eigh
from scipy.signal import butter, sosfiltfilt

FS = 160
BAND = (8.0, 30.0)
FILTER_ORDER = 5
CHANNELS = [ FC5 ,  FC3 ,  FC1 ,  FCz ,  FC2 ,  FC4 ,  FC6 ,
             C5 ,  C3 ,  C1 ,  Cz ,  C2 ,  C4 ,  C6 ,
             CP5 ,  CP3 ,  CP1 ,  CPz ,  CP2 ,  CP4 ,  CP6 ]
TMIN, TMAX = 0.5, 2.5            # seconds after cue onset
IMAGERY_RUNS = [4, 8, 12]
EXECUTION_RUNS = [3, 7, 11]
SEED = 2601
M_VALUES = list(range(1, 11))    # 2m <= 21 channels
K_VALUES = list(range(1, 100, 2))
N_FOLDS = 5


def bandpass(data, fs=FS):
    sos = butter(FILTER_ORDER, BAND, btype= band , fs=fs, output= sos )
    return sosfiltfilt(sos, data, axis=-1)


def subject_kfold(subjects, n_splits=N_FOLDS, seed=SEED):
    uniq = np.unique(subjects)
    rng = np.random.default_rng(seed)
    rng.shuffle(uniq)
    folds = []
    for group in np.array_split(uniq, n_splits):
        va = np.isin(subjects, group)
        folds.append((np.where(~va)[0], np.where(va)[0]))
    return folds


def trial_covariances(X):
    X = X - X.mean(axis=2, keepdims=True)
    C = np.einsum( ict,idt->icd , X, X) / X.shape[2]
    return C / np.trace(C, axis1=1, axis2=2)[:, None, None]    # trace
       normalization


def csp_filters(X, y):
    C = trial_covariances(X)
    R0, R1 = C[y == 0].mean(axis=0), C[y == 1].mean(axis=0)
    evals, evecs = eigh(R0, R0 + R1)
    return evecs[:, np.argsort(evals)[::-1]]


def component_variances(X, W):
    return np.einsum( cf,ict->ift , W, X).var(axis=2)


def features_from_variances(v, m):
```

```
    sel = np.concatenate([v[:, :m], v[:, -m:]], axis=1)
    return np.log(sel / sel.sum(axis=1, keepdims=True))


def knn_predict_multi(f_tr, y_tr, f_te, k_values):
    d = (np.sum(f_te**2, axis=1)[:, None] + np.sum(f_tr**2, axis=1)[None,
        :]
         - 2 * f_te @ f_tr.T)
    nn = y_tr[np.argsort(d, axis=1)[:, :max(k_values)]]
    return {k: (nn[:, :k].mean(axis=1) > 0.5).astype(int) for k in k_values
        }


def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -250, 250)))


def train_lr(F, y, lr=0.1, iters=2000, lam=0.0):
    w, b, n = np.zeros(F.shape[1]), 0.0, len(y)
    for _ in range(iters):
        g = sigmoid(F @ w + b) - y
        w -= lr * (F.T @ g + lam * w) / n
        b -= lr * g.mean()
    return w, b


def predict_lr(F, w, b):
    return (sigmoid(F @ w + b) >= 0.5).astype(int)


def train_lda(F, y, alpha=0.1):
    mu0, mu1 = F[y == 0].mean(axis=0), F[y == 1].mean(axis=0)
    S = (np.cov(F[y == 0].T) + np.cov(F[y == 1].T)) / 2
    S = (1 - alpha) * S + alpha * np.eye(F.shape[1])
    w = np.linalg.solve(S, mu1 - mu0)
    b = -0.5 * (mu0 + mu1) @ w
    return w, b


def predict_lda(F, w, b):
    return (F @ w + b >= 0).astype(int)
```

Code 2 (01_preprocess.py)

```
import json
import numpy as np
import matplotlib.pyplot as plt
import mne
from mne.datasets import eegbci
from utilities import FS, CHANNELS, TMIN, TMAX, IMAGERY_RUNS,
    EXECUTION_RUNS, SEED, bandpass

mne.set_log_level( ERROR )
ALL_SUBJECTS = range(1, 110)
TEST_FRACTION = 0.2
```

```
def load_run(subject, run):
    path = eegbci.load_data(subject, [run], update_path=True)[0]
    raw = mne.io.read_raw_edf(path, preload=True)
    eegbci.standardize(raw)
    return raw


def extract_trials(raw):
    missing = [c for c in CHANNELS if c not in raw.ch_names]
    if missing:
        raise ValueError(f Missing channels {missing}. Available: {raw.
            ch_names} )
    data = raw.get_data(picks=CHANNELS) * 1e6            # V -> uV
    data = data - data.mean(axis=1, keepdims=True)
    filtered = bandpass(data)
    events, _ = mne.events_from_annotations(raw, event_id={ T1 : 1,  T2 :
        2})
    start, length = int(TMIN * FS), int((TMAX - TMIN) * FS)
    X, y = [], []
    for onset, _, code in events:
        s = onset - raw.first_samp + start
        if s + length <= filtered.shape[1]:
            X.append(filtered[:, s:s + length])
            y.append(0 if code == 1 else 1)              # T1 = left fist,
                T2 = right fist
    return X, y


def process_subject(subject, runs):
    X, y = [], []
    for run in runs:
        raw = load_run(subject, run)
        if raw.info[ sfreq ] != FS:
            return None
        Xr, yr = extract_trials(raw)
        X += Xr
        y += yr
    if len(y) == 0:
        return None
    return np.array(X, dtype=np.float32), np.array(y)


def stack(results, ids):
    X = np.concatenate([results[s][0] for s in ids])
    y = np.concatenate([results[s][1] for s in ids])
    subj = np.concatenate([np.full(len(results[s][1]), s) for s in ids])
    return X, y, subj


imagery, excluded = {}, []
for s in ALL_SUBJECTS:
    out = process_subject(s, IMAGERY_RUNS)
    if out is None:
        excluded.append(s)
        print(f Subject {s:3d}: excluded )
    else:
        imagery[s] = out
        print(f Subject {s:3d}: {len(out[1])} imagery trials )
```

```
valid = np.array(sorted(imagery))
perm = np.random.default_rng(SEED).permutation(valid)
n_test = int(round(TEST_FRACTION * len(valid)))
test_ids = sorted(int(s) for s in perm[:n_test])
train_ids = sorted(int(s) for s in perm[n_test:])

execution = {}
for s in test_ids:
    out = process_subject(s, EXECUTION_RUNS)
    if out is not None:
        execution[s] = out

for name, res, ids in [( train , imagery, train_ids),
                       ( test , imagery, test_ids),
                       ( exec , execution, sorted(execution))]:
    X, y, subj = stack(res, ids)
    np.savez_compressed(f data_{name}.npz , X=X, y=y, subject=subj)
    print(f {name}: {X.shape[0]} trials, {len(ids)} subjects,
          f shape {X.shape}, right-hand ratio {y.mean():.3f} )

with open( split.json ,  w ) as f:
    json.dump({ train : train_ids,  test : test_ids,  excluded : excluded,
                channels : CHANNELS,  window_s : [TMIN, TMAX]}, f, indent
                  =2)
print(f Excluded subjects: {excluded} )

# Figure 1: raw vs. filtered, same electrode (C3)
raw = load_run(1, IMAGERY_RUNS[0])
c3 = raw.get_data(picks=[ C3 ])[0] * 1e6
c3_filt = bandpass(c3 - c3.mean())
seg, t = slice(800, 800 + FS), np.arange(FS) / FS

fig, ax = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
ax[0].plot(t, c3[seg], color= firebrick , label= Raw signal )
ax[0].set_title( Graph A: Raw EEG Signal (C3) - Subject 1 , fontsize=16,
    fontweight= bold )
ax[1].plot(t, c3_filt[seg], color= forestgreen , label= Filtered (8-30 Hz,
    zero-phase) )
ax[1].set_title( Graph B: Filtered EEG Signal (C3) , fontsize=16,
    fontweight= bold )
ax[1].set_xlabel( Time (seconds) , fontsize=12)
for a in ax:
    a.set_ylabel(r Amplitude ($\mu V$) , fontsize=12)
    a.grid(True, alpha=0.3)
    a.legend(loc= upper right )
plt.tight_layout()
plt.savefig( preprocessing_comparison.png , dpi=300)
plt.close()
```

Code 3 (02_cross_validation.py)

```
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binomtest, wilcoxon
```

```
from utilities import (csp_filters, component_variances,
    features_from_variances,
                       knn_predict_multi, train_lr, predict_lr, train_lda,
                          predict_lda)

with open( best_params.json ) as f:
    best = json.load(f)


def load(name):
    d = np.load(f data_{name}.npz )
    return d[ X ].astype(np.float64), d[ y ], d[ subject ]


Xtr, ytr, _ = load( train )
sets = { MI : load( test ),  ME : load( exec )}

W = csp_filters(Xtr, ytr)                                  # final CSP on all
    training subjects
v_tr = component_variances(Xtr, W)
v_te = {c: component_variances(sets[c][0], W) for c in sets}


def fit_predict(model, f_tr, f_te):
    if model ==  knn :
        k = best[ knn ][ k ]
        return knn_predict_multi(f_tr, ytr, f_te, [k])[k]
    if model ==  lr :
        return predict_lr(f_te, *train_lr(f_tr, ytr))
    return predict_lda(f_te, *train_lda(f_tr, ytr))


results = {}
for model in [ knn ,  lr ,  lda ]:
    m = best[model][ m ]
    f_tr = features_from_variances(v_tr, m)
    results[model] = { m : m}
    for c, (_, y, subj) in sets.items():
        pred = fit_predict(model, f_tr, features_from_variances(v_te[c], m)
            )
        n, k = len(y), int(np.sum(pred == y))
        per_subj = {int(s): float(np.mean(pred[subj == s] == y[subj == s]))
                    for s in np.unique(subj)}
        accs = np.array(list(per_subj.values()))
        results[model][c] = {
             accuracy : k / n,  correct : k,  n : n,
             z : (k - n / 2) / np.sqrt(n / 4),
             p_binomial_one_sided : float(binomtest(k, n, 0.5, alternative=
                 greater ).pvalue),
             subject_mean : float(accs.mean()),  subject_sd : float(accs.
               std(ddof=1)),
             subject_min : float(accs.min()),  subject_max : float(accs.max
               ()),
             per_subject : per_subj,
        }
    mi_s, me_s = results[model][ MI ][ per_subject ], results[model][ ME ][
        per_subject ]
    common = sorted(set(mi_s) & set(me_s))
```

```
    diff = np.array([me_s[s] - mi_s[s] for s in common])
    stat, p = wilcoxon(diff)
    results[model][ ME_vs_MI_paired ] = { mean_diff : float(diff.mean()),
                                          wilcoxon_stat : float(stat),  p :
                                             float(p),
                                          n_subjects : len(common)}

with open( results.json ,  w ) as f:
    json.dump(results, f, indent=2, default=float)

print(f {'Model':6s} {'Cond':4s} {'Acc':>7s} {'k/n':>10s} {'Z':>6s} {'p':>9
    s} {'Subj mean sd':>14s} )
for model in [ knn ,  lr ,  lda ]:
    for c in [ MI ,  ME ]:
        r = results[model][c]
        print(f {model:6s} {c:4s} {100 * r['accuracy']:6.2f}% {r['correct
            ']:4d}/{r['n']:<5d}
              f {r['z']:6.2f} {r['p_binomial_one_sided']:9.2e}
              f {100 * r['subject_mean']:5.1f}  {100 * r['subject_sd']:4.1f
                 } )
    pm = results[model][ ME_vs_MI_paired ]
    print(f {model:6s} ME-MI paired: diff {100 * pm['mean_diff']:+.2f} pts,

          f Wilcoxon p = {pm['p']:.3f} (n = {pm['n_subjects']}) )
    print(f {model:6s} CV acc at chosen params: {100 * best[model]['cv_acc
        ']:.2f}%\n )

# Figure: per-subject accuracy
fig, axes = plt.subplots(3, 1, figsize=(12, 12), sharey=True)
for ax, model in zip(axes, [ knn ,  lr ,  lda ]):
    mi_s, me_s = results[model][ MI ][ per_subject ], results[model][ ME ][
        per_subject ]
    subs = sorted(mi_s)
    x = np.arange(len(subs))
    ax.scatter(x, [mi_s[s] for s in subs], label= Imagery (MI) , zorder=3)
    ax.scatter(x, [me_s.get(s, np.nan) for s in subs], marker= s , label=
       Execution (ME) , zorder=3)
    ax.axhline(0.5, color= gray , linestyle= -- , label= Chance )
    ax.set_xticks(x)
    ax.set_xticklabels(subs, rotation=90)
    ax.set_title(model.upper())
    ax.set_xlabel( Test subject )
    ax.grid(True, alpha=0.3)
    ax.set_ylabel( Accuracy )
axes[0].legend()
plt.tight_layout()
plt.savefig( per_subject_accuracy.png , dpi=300)
plt.close()
```

Code 4 (03_test.py)

```
import json
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binomtest, wilcoxon
from utilities import (csp_filters, component_variances,
    features_from_variances,
```

```
                        knn_predict_multi, train_lr, predict_lr, train_lda,
                            predict_lda)

with open( best_params.json ) as f:
    best = json.load(f)


def load(name):
    d = np.load(f data_{name}.npz )
    return d[ X ].astype(np.float64), d[ y ], d[ subject ]


Xtr, ytr, _ = load( train )
sets = { MI : load( test ),  ME : load( exec )}

W = csp_filters(Xtr, ytr)                                    # final CSP on all
    training subjects
v_tr = component_variances(Xtr, W)
v_te = {c: component_variances(sets[c][0], W) for c in sets}


def fit_predict(model, f_tr, f_te):
    if model ==  knn :
        k = best[ knn ][ k ]
        return knn_predict_multi(f_tr, ytr, f_te, [k])[k]
    if model ==  lr :
        return predict_lr(f_te, *train_lr(f_tr, ytr))
    return predict_lda(f_te, *train_lda(f_tr, ytr))


results = {}
for model in [ knn ,  lr ,  lda ]:
    m = best[model][ m ]
    f_tr = features_from_variances(v_tr, m)
    results[model] = { m : m}
    for c, (_, y, subj) in sets.items():
        pred = fit_predict(model, f_tr, features_from_variances(v_te[c], m)
            )
        n, k = len(y), int(np.sum(pred == y))
        per_subj = {int(s): float(np.mean(pred[subj == s] == y[subj == s]))
                    for s in np.unique(subj)}
        accs = np.array(list(per_subj.values()))
        results[model][c] = {
             accuracy : k / n,  correct : k,  n : n,
             z : (k - n / 2) / np.sqrt(n / 4),
             p_binomial_one_sided : float(binomtest(k, n, 0.5, alternative=
                greater ).pvalue),
             subject_mean : float(accs.mean()),  subject_sd : float(accs.
               std(ddof=1)),
             subject_min : float(accs.min()),  subject_max : float(accs.max
               ()),
             per_subject : per_subj,
        }
    mi_s, me_s = results[model][ MI ][ per_subject ], results[model][ ME ][
         per_subject ]
    common = sorted(set(mi_s) & set(me_s))
    diff = np.array([me_s[s] - mi_s[s] for s in common])
    stat, p = wilcoxon(diff)
```

```
    results[model][ ME_vs_MI_paired ] = { mean_diff : float(diff.mean()),
                                          wilcoxon_stat : float(stat),  p :
                                             float(p),
                                          n_subjects : len(common)}

with open( results.json ,  w ) as f:
    json.dump(results, f, indent=2, default=float)

print(f {'Model':6s} {'Cond':4s} {'Acc':>7s} {'k/n':>10s} {'Z':>6s} {'p':>9
    s} {'Subj mean sd':>14s} )
for model in [ knn ,  lr ,  lda ]:
    for c in [ MI ,  ME ]:
        r = results[model][c]
        print(f {model:6s} {c:4s} {100 * r['accuracy']:6.2f}% {r['correct
            ']:4d}/{r['n']:<5d}
              f {r['z']:6.2f} {r['p_binomial_one_sided']:9.2e}
              f {100 * r['subject_mean']:5.1f}  {100 * r['subject_sd']:4.1f
                 } )
    pm = results[model][ ME_vs_MI_paired ]
    print(f {model:6s} ME-MI paired: diff {100 * pm['mean_diff']:+.2f} pts,

          f Wilcoxon p = {pm['p']:.3f} (n = {pm['n_subjects']}) )
    print(f {model:6s} CV acc at chosen params: {100 * best[model]['cv_acc
        ']:.2f}%\n )

# Figure: per-subject accuracy
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)
for ax, model in zip(axes, [ knn ,  lr ,  lda ]):
    mi_s, me_s = results[model][ MI ][ per_subject ], results[model][ ME ][
        per_subject ]
    subs = sorted(mi_s)
    x = np.arange(len(subs))
    ax.scatter(x, [mi_s[s] for s in subs], label= Imagery (MI) , zorder=3)
    ax.scatter(x, [me_s.get(s, np.nan) for s in subs], marker= s , label=
       Execution (ME) , zorder=3)
    ax.axhline(0.5, color= gray , linestyle= -- , label= Chance )
    ax.set_xticks(x)
    ax.set_xticklabels(subs, rotation=90)
    ax.set_title(model.upper())
    ax.set_xlabel( Test subject )
    ax.grid(True, alpha=0.3)
axes[0].set_ylabel( Accuracy )
axes[0].legend()
plt.tight_layout()
plt.savefig( per_subject_accuracy.png , dpi=300)
plt.close()
```

Code 5 (04_mne_baseline.py)

```
import json
import numpy as np
from mne.decoding import CSP
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.pipeline import make_pipeline

with open( best_params.json ) as f:
    m = json.load(f)[ lda ][ m ]
```

```
def load(name):
    d = np.load(f data_{name}.npz )
    return d[ X ].astype(np.float64), d[ y ]


Xtr, ytr = load( train )
clf = make_pipeline(CSP(n_components=2 * m, log=True),
   LinearDiscriminantAnalysis())
clf.fit(Xtr, ytr)
for name, label in [( test ,  MI ), ( exec ,  ME )]:
    X, y = load(name)
    print(f MNE CSP + sklearn LDA ({label}): {100 * clf.score(X, y):.2f}% )
```